Bilgisayar Programlama

Döngüler (Loops)

for, while, do-while, break, continue

İçindekiler

1. Döngü Kavramına Giriş
   - Döngü Nedir, Neden Gereklidir?
   - Döngülerin Temel Bileşenleri
2. for Döngüsü Detaylı İncelenmesi
   - Sözdizimi, Çalışma Mekanizması, Esnek Kullanım
3. İç İçe for Döngüleri (Nested Loops)
   - Çalışma Prensibi, Çarpım Tablosu, Karmaşıklık Analizi
4. while ve do-while Döngüleri
5. Döngü Kontrol Kavramları ve Algoritma Desenleri
6. break ve continue Deyimleri
7. Uygulamalar

BÖLÜM 1

Döngü Kavramına Giriş

1.1. Tekrarlı Akış (Repetition) Nedir?

Geçmiş haftalarda program akışının üç temel mekanizma ile yönetildiğini öğrenmiştik:

1. **Sıralı Akış (Sequential)**: Komutlar yukarıdan aşağıya sırayla çalışır.
2. **Seçimli Akış (Selection)**: `if`, `switch` gibi yapılarla dallanma yapılır.
3. **Tekrarlı Akış (Repetition)**: Belirli bir kod bloğu, koşul sağlandığı sürece döngüsel olarak çalıştırılır.

Bu hafta **3. mekanizma** olan **tekrarlı akışı** (döngüleri) ele alacağız.

1.1.1. Tekrarlı Akış (Repetition) Nedir?

Tekrarlı akış, bir kod bloğunun belirli bir koşul sağlandığı sürece tekrar tekrar çalıştırılmasıdır. Döngü, bu akışı sağlayan programlama yapılarına verilen isimdir.

Aslında döngüler günlük hayatımızda her yerde karşımıza çıkar. Farkında olmadan her gün onlarca döngü yaşarız.

Gerçek Hayattan Döngü Örnekleri

Günlük Hayat Örneği	Başlangıç	Koşul	Güncelleme
⏰ Çalar saat	Alarm çalar	Erteleme tuşuna basıldı mı?	Her 5 dk'da tekrar çal
🏭 Fabrika bant sistemi	Ürün banda girer	Bant üzerinde ürün var mı?	Sonraki ürünü al
🚗 Araç yıkama makinesi	Araç girer	Tüm aşamalar bitti mi?	Sonraki aşamaya geç (su → sabun → durulama → kurulama)
🍽️ Bulaşık makinesi	Program başlar	Seçilen program bitti mi?	Sonraki döngüye geç (ön yıkama → ana yıkama → durulama)
🏋️ Spor antrenmanı	1. tekrar	12 tekrar tamamlandı mı?	Tekrar sayısını artır

1.2. Döngüler Neden Gereklidir?

Döngüler olmadan tekrarlı işlemleri yapmak için aynı kodu defalarca yazmak gerekir.

Problemi düşünelim: Ekrana 1'den 1000'e kadar sayıları yazdırmak istiyorsunuz.

❌ Döngü olmadan: 1000 satır `printf` yazmak gerekir → **Pratik değil!**

✅ Döngü ile: **Sadece 3 satır** kod yeterlidir:

```
1  for (int i = 1; i <= 1000; i++) {
2      printf("%d\n", i);
3  }
```

1.3. Mühendislik Sistemlerinde Döngüler

Gerçek dünya mühendislik uygulamalarında döngüler her yerdedir:

Uygulama Alanı	Döngü Kullanım Senaryosu
Sensör Sistemleri	Bir sensörden saniyede 1000+ kez veri okuma
Veri İşleme	Bir dizinin tüm elemanlarını analiz etme
Motor Kontrolü	Hızı 0'dan max'a kademeli olarak artırma (rampa)
Kontrol Algoritmaları	PID algoritmasını sürekli döngüde çalıştırma
Görüntü İşleme	Bir görüntünün her pikselini tarama

1.4. Döngülerin Üç Temel Bileşeni

Her döngü yapısı bu üç bileşene dayanır. Birinin bile eksik veya hatalı olması sorunlara yol açar:

1. **Başlangıç (Initialization):** Döngü sayacının veya kontrol değişkeninin ilk değerinin atanması. *Sadece bir kez* çalışır.
2. **Koşul (Condition):** Döngünün devam edip etmeyeceğini belirleyen mantıksal test. Her turda kontrol edilir.
3. **Güncelleme (Update):** Her iterasyonda sayacın değiştirilmesi. Güncelleme eksikse sonsuz döngü oluşur!

1.5. Üç Bileşenin Görselleştirilmesi

```
1
2   ┌──────────────────────┐
    │ 1. BAŞLANGIÇ (bir kez) │  ← int i = 0;
3   └──────────────────────┘
4            │
             ▼
5                                  Hayır (false)
6   ┌──────────────────────┐
    │ 2. KOŞUL KONTROLÜ      │ ──────────────────────► Döngüden Çık
7   │    (her turda)         │
    └──────────────────────┘
8
9            │ Evet (true)
             ▼
10
11
12  ┌──────────────────────┐
    │ Döngü Gövdesi Çalışır  │  ← printf(...);
13  └──────────────────────┘
14           │
             ▼
15
16  ┌──────────────────────┐
    │ 3. GÜNCELLEME          │  ← i++;
17  └──────────────────────┘
18           │
             └──────────────► Adım 2'ye geri dön
19
```

1.6. C Dilinde Üç Döngü Yapısı

C dilinde bu üç bileşen, üç farklı döngü yapısında farklı biçimlerde organize edilir:

Döngü Tipi	Kontrol Türü	Kullanım Alanı
`for`	Giriş kontrollü	Tekrar sayısı **önceden belli** ise
`while`	Giriş kontrollü	Tekrar sayısı **önceden belirsiz** ise
`do-while`	Çıkış kontrollü	En az **1 kez çalışması garanti** olmalı ise

BÖLÜM 2

`for` Döngüsü

2.1. `for` Döngüsünün Sözdizimi (Syntax)

C dilinde en sık kullanılan ve en yapılandırılmış döngü biçimidir. Üç bileşenin tamamı tek satırda, parantez `()` içinde yan yana yazılır:

```
for (başlangıç; koşul; güncelleme) {
    // Döngü gövdesi (body)
    // Koşul doğru olduğu sürece tekrarlanır
}
```

Kritik kural: Üç bileşen **noktalı virgül** (`;`) ile ayrılır. Bileşenlerden biri veya birkaçı boş bırakılabilir, ancak noktalı virgüller **her zaman** yazılmalıdır.

2.2. for Döngüsünün Çalışma Mekanizması

Adım adım yürütme sırası:

1. **Başlangıç ifadesi** yalnızca bir kez çalıştırılır (`int i = 0`)
2. **Koşul ifadesi** kontrol edilir:
   - **Doğru (true / sıfırdan farklı)** → Döngü gövdesi çalıştırılır
   - **Yanlış (false / 0)** → Döngü **sonlanır**, gövde atlanır
3. Gövde tamamlandıktan sonra **güncelleme ifadesi** çalışır (`i++`)
4. **Adım 2'ye** geri dönülür (Başlangıç adımına **değil!**)

⚠️ **Sık yapılan kavram hatası:** Güncelleme sonrası başlangıca değil, doğrudan koşul kontrolüne dönülür.

Neden Başlangıca Dönülmez?

Eğer her turda başlangıca (`int i = 0`) dönülseydi, sayaç her seferinde sıfırlanır ve döngü **sonsuza kadar** çalışırdı.

```
for (int i = 0;  i < 5;  i++)
     ─────────   ──────  ────
       │           │       │
       │           │       └── 3. Güncelleme (her turda)
       │           └─────────── 2. Koşul (her turda kontrol)
       └──────────────────────── 1. Başlangıç (SADECE 1 KEZ)

Akış:  1 → 2 → gövde → 3 → 2 → gövde → 3 → 2 → ...
                                  ▲
                                  │
                     3'ten sonra 1'e DEĞİL, 2'ye dönülür!
```

2.3. Örnek: Temel Sayaç Döngüsü

1'den 10'a kadar sayıları ekrana yazdırma:

```
1  for (int i = 1; i <= 10; i++) {
2      printf("%d ", i);
3  }
4  // Çıktı: 1 2 3 4 5 6 7 8 9 10
```

İzleme Tablosu (Trace Table):

Adım	`i` değeri	Koşul `i<=10`	Çıktı	Güncelleme
1	1	✅ True	`1`	i→2
2	2	✅ True	`2`	i→3
...	...	...	...	...
10	10	✅ True	`10`	i→11
11	11	❌ False	—	Döngü biter

2.4. Örnek: Azalan Sayaç

Geri sayım yapmak için başlangıç büyük, koşul küçüğe doğru, güncelleme `i--` şeklinde tanımlanır:

```
1  // 10'dan 1'e geri sayım
2  for (int i = 10; i >= 1; i--) {
3      printf("%d ", i);
4  }
5  // Çıktı: 10 9 8 7 6 5 4 3 2 1
```

Dikkat: Koşul `i >= 1` şeklindedir. `i > 1` yazsaydık 1 değeri **yazdırılmazdı** (Off-by-one hatası!).

⚠️ Off-by-One (Sınır İhlali) Hatası

Döngü koşullarında `<` ile `<=` veya `>` ile `>=` karıştırmak en sık yapılan mantık hatasıdır:

```
// ❌ HATALI: 1 değeri yazdırılmaz!
for (int i = 10; i > 1; i--) {
    printf("%d ", i);
}
// Çıktı: 10 9 8 7 6 5 4 3 2   ← 1 EKSİK!

// ✅ DOĞRU: 1 dahil yazdırılır
for (int i = 10; i >= 1; i--) {
    printf("%d ", i);
}
// Çıktı: 10 9 8 7 6 5 4 3 2 1  ← TAMAM
```

Kural: Döngü yazarken **sınır değerlerini** her zaman test edin: *İlk değer dahil mi? Son değer dahil mi?*

2.5. Örnek: Farklı Artış Miktarları

Güncelleme kısmında `i++` yerine farklı artış/azalış değerleri kullanılabilir:

```
1   // Çift sayılar: 0'dan 20'ye 2'şer artarak
2   for (int i = 0; i <= 20; i += 2) {
3       printf("%d ", i);
4   }
5   // Çıktı: 0 2 4 6 8 10 12 14 16 18 20
6
7   // 3'er artarak
8   for (int i = 0; i <= 30; i += 3) {
9       printf("%d ", i);
10  }
11  // Çıktı: 0 3 6 9 12 15 18 21 24 27 30
```

2.6. for Döngüsünün Esnek Kullanımı

`for` döngüsünün üç bileşeninin her biri **opsiyoneldir**. Ancak noktalı virgüller her zaman yazılmalıdır. Bu esneklik 4 farklı biçimde karşımıza çıkar:

Esneklik 1: Başlangıç Dışarıda

Sayaç değişkeni döngüden önce tanımlanmışsa, başlangıç kısmı boş bırakılabilir:

```c
int i = 1;            // Başlangıç dışarıda tanımlandı
for (; i <= 10; i++) {
    printf("%d ", i);
}
// i değişkeni döngüden sonra da erişilebilir durumda kalır
printf("\nDöngü sonrası i = %d", i); // i = 11
```

Not: Bu durumda `i` değişkeni döngü dışında da geçerlidir (scope farkı).

Esneklik 2: Güncelleme Döngü Gövdesinde

Güncelleme ifadesi parantez içinde değil, gövde içinde de yazılabilir:

```
for (int i = 1; i <= 10; ) {  // Güncelleme kısmı boş
    printf("%d ", i);
    i++;  // Güncelleme gövdede yapılıyor
}
```

⚠️ **Tehlike:** Güncellemeyi gövde içine yazdığınızda, kompleks kodlarda güncellemeyi unutma riski artar.

Esneklik 3: Sonsuz Döngü (Infinite Loop)

Üç bileşenin tamamı boş bırakılırsa döngü sonsuza kadar çalışır. Koşul yoksa varsayılan değer her zaman `true` kabul edilir:

```
1  for (;;) {
2      printf("Bu mesaj durmadan tekrarlanır!\n");
3      // Kontrollü çıkış için break kullanılmalıdır:
4      // if (bir_kosul) break;
5  }
```

Gömülü sistemlerde (embedded systems) ana döngü genellikle sonsuz döngü olarak tasarlanır.

Esneklik 4: Virgül Operatörü ile Çoklu Değişken

Başlangıç ve güncelleme kısımlarında virgül (`,`) operatörü ile birden fazla işlem yapılabilir:

```
for (int i = 0, j = 10; i <= j; i++, j--) {
    printf("i=%d, j=%d\n", i, j);
}
```

```
Çıktı:
i=0, j=10
i=1, j=9
i=2, j=8
i=3, j=7
i=4, j=6
i=5, j=5
```

2.6. for Döngüsü Kullanım Örnekleri

🔤 Karakter Döngüsü (char ile for)

C dilinde `char` tipi aslında bir tamsayıdır. Bu sayede harfler üzerinde de döngü kurulabilir:

```
// Alfabeyi yazdır
for (char c = 'A'; c <= 'Z'; c++) {
    printf("%c ", c);
}
// Çıktı: A B C D E F G H I J K L M N O P Q R S T U V W X Y Z
```

`'A'` ASCII tablosunda 65, `'Z'` ise 90 değerine karşılık gelir. `c++` her adımda ASCII kodunu 1 artırarak bir sonraki harfe geçer.

🔢 Ondalıklı (float/double) Sayaç

Sayaç değişkeni `int` olmak zorunda değildir. `double` ile kayan noktalı adımlar atılabilir:

```
// 0.0'dan 1.0'a 0.1 adımlarla
for (double t = 0.0; t <= 1.0; t += 0.1) {
    printf("%.1f ", t);
}
// Çıktı: 0.0 0.1 0.2 0.3 0.4 0.5 0.6 0.7 0.8 0.9 1.0
```

⚠️ **Dikkat:** Kayan nokta aritmetiğinde hassasiyet (precision) hataları oluşabilir. `0.1 + 0.1 + 0.1` tam olarak `0.3` olmayabilir! Kritik ölçümlerde tamsayı sayaç tercih edilmelidir.

🧮 Bit Düzeyinde Tarama (Bitwise Scan)

Bir sayının ikili (binary) temsilindeki bitleri for döngüsüyle tek tek inceleyebilirsiniz. Gömülü sistemlerde register okuma/yazma işlemlerinde sıkça kullanılır:

```c
unsigned int x = 0xAB; // İkili: 10101011
printf("0x%X sayısının bitleri:\n", x);

for (int bit = 7; bit >= 0; bit--) {
    printf("Bit %d: %d\n", bit, (x >> bit) & 1);
}
// Çıktı: Bit 7: 1, Bit 6: 0, Bit 5: 1, ... Bit 0: 1
```

`(x >> bit) & 1` ifadesi belirtilen konumdaki bitin 0 mı 1 mi olduğunu verir.

BÖLÜM 3

İç İçe `for` Döngüleri (Nested Loops)

3.1. İç İçe Döngü Nedir?

Bir döngünün gövdesinde başka bir döngü bulunması yapısına **iç içe döngü (nested loop)** denir.

Temel Kural: İç döngü, dış döngünün **her bir iterasyonunda** baştan sona tamamen çalışır.

Kullanım alanları:

- **Matris / 2B dizi** işlemleri (satır × sütun tarama)
- **Tablo** oluşturma ve **desen çizme**
- **Sıralama** algoritmaları (Bubble Sort, Selection Sort)
- **Kombinasyon** ve **permütasyon** hesaplama

3.2. Çalışma Prensibi - Kod ve İzleme

```c
for (int i = 1; i <= 3; i++) {          // Dış döngü
    printf("Dış: i = %d\n", i);
    for (int j = 1; j <= 2; j++) {      // İç döngü
        printf("  İç: j = %d\n", j);
    }
}
```

Çıktı:

```
Dış: i = 1          ← Dış döngü 1. tur
  İç: j = 1         ← İç döngü tamamen çalışır
  İç: j = 2
Dış: i = 2          ← Dış döngü 2. tur
  İç: j = 1         ← İç döngü BAŞTAN çalışır
  İç: j = 2
Dış: i = 3          ← Dış döngü 3. tur
  İç: j = 1
  İç: j = 2
```

3.3. Toplam İterasyon Sayısının Hesaplanması

İç içe döngülerde toplam yürütme sayısı, döngü sayılarının **çarpımıdır**:

- Dış döngü: n kez → İç döngü: m kez → Toplam: n × m kez

Önceki örnekte: Dış = 3, İç = 2 → Toplam = **6 kez** iç gövde çalışır.

Üç katlı iç içe döngü:

```
1  for (int i = 0; i < 10; i++)        // 10 kez
2      for (int j = 0; j < 20; j++)    // 20 kez
3          for (int k = 0; k < 5; k++) // 5 kez
4              // Toplam: 10 × 20 × 5 = 1000 kez
```

3.4. Çarpım Tablosu Örneği

```c
// Başlık satırı
printf("     ");
for (int j = 1; j <= 10; j++) printf("%4d", j);
printf("\n     ");
for (int j = 1; j <= 10; j++) printf("----");
printf("\n");

// Tablo gövdesi
for (int i = 1; i <= 10; i++) {
    printf("%2d | ", i);
    for (int j = 1; j <= 10; j++) {
        printf("%4d", i * j);
    }
    printf("\n");
}
```

Bu kodda dış döngü **satırları**, iç döngü **sütunları** temsil eder.

3.5. Desen Çizimi - Sağ Üçgen

İç içe döngülerin en klasik eğitim örneği olan yıldız desenleri:

```c
int n = 5;
for (int i = 1; i <= n; i++) {        // Satır sayısı
    for (int j = 1; j <= i; j++) {    // Her satırdaki yıldız sayısı = i
        printf("* ");
    }
    printf("\n");
}
```

```
Çıktı:
*
* *
* * *
* * * *
* * * * *
```

Gözlem: i. satırda tam olarak `i` adet yıldız basılır.

3.6. Desen Çizimi - Piramit

Piramit için her satırda önce boşluk, sonra yıldız basılır:

```c
int n = 5;
for (int i = 1; i <= n; i++) {
    // Boşluklar: (n - i) adet
    for (int j = 1; j <= n - i; j++)
        printf(" ");
    // Yıldızlar: (2*i - 1) adet
    for (int j = 1; j <= 2 * i - 1; j++)
        printf("*");
    printf("\n");
}
```

```
    *
   ***
  *****
 *******
*********
```

3.7. Floyd Üçgeni

Her hücreye ardışık artan bir sayı yerleştirilir:

```c
int sayi = 1;
for (int i = 1; i <= 5; i++) {
    for (int j = 1; j <= i; j++) {
        printf("%3d ", sayi++);
    }
    printf("\n");
}
```

```
  1
  2   3
  4   5   6
  7   8   9  10
 11  12  13  14  15
```

3.8. Karmaşıklık Analizi (Big-O Notasyonu)

İç içe döngülerin **zaman karmaşıklığı** analizi:

Yapı	Karmaşıklık	Örnek
Tek döngü (n kez)	$O(n)$	Diziyi tarama
İki iç içe döngü (n × n)	$O(n^2)$	Bubble Sort, matris tarama
Üç iç içe döngü (n × n × n)	$O(n^3)$	Matris çarpımı

Pratik Anlam: n = 1000 için:

- $O(n)$ → **1.000** işlem
- $O(n^2)$ → **1.000.000** işlem
- $O(n^3)$ → **1.000.000.000** işlem → Ciddi performans sorunu!

BÖLÜM 4

while ve do-while Döngüleri

4.1. while Döngüsünün Sözdizimi

`while` döngüsünde bileşenler **ayrıdır** (for'daki gibi tek parantezde değildir):

```
1  // Başlangıç (döngüden önce)
2  int i = 1;
3
4  while (koşul) {        // Koşul (her turda kontrol edilir)
5      // Döngü gövdesi
6      printf("%d ", i);
7      i++;                // Güncelleme (gövde içinde)
8  }
```

Giriş kontrollü (pre-test) döngüdür: Koşul daha ilk seferde yanlışsa, gövde hiç çalışmaz.

4.2. for vs while Karşılaştırması

Aynı iş, iki farklı sözdizimi:

```c
// for ile — Üç bileşen tek satırda
for (int i = 1; i <= 10; i++) {
    printf("%d ", i);
}

// while ile — Bileşenler dağınık
int i = 1;              // Başlangıç → dışarıda
while (i <= 10) {       // Koşul → parantezde
    printf("%d ", i);
    i++;                // Güncelleme → gövdede
}
```

Kriter	`for` tercih et	`while` tercih et
Tekrar sayısı	Belli	Belirsiz
Yapı	Sayaç kontrollü	Koşul/olay kontrollü

4.3. while Örneği: Basamak Sayısını Bulma

Bir tamsayının kaç basamaktan oluştuğunu hesaplama (tekrar sayısı önceden bilinmez):

```c
int sayi = 12345;
int basamak = 0;
int gecici = sayi;

while (gecici != 0) {
    gecici /= 10;    // Son basamağı at (tamsayı bölme)
    basamak++;       // Sayacı artır
}
printf("%d sayısı %d basamaklıdır.\n", sayi, basamak);
// Çıktı: 12345 sayısı 5 basamaklıdır.
```

Açıklama:

Adım	`gecici`	`basamak`
Başlangıç	12345	0
1	1234	1
2	123	2
3	12	3
4	1	4
5	0	5 → Döngü biter

4.4. while Örneği: Sayı Tahmin Oyunu

```c
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

int main() {
    srand(time(NULL));
    int hedef = rand() % 100 + 1;  // 1-100 arası rastgele sayı
    int tahmin, deneme = 0;

    printf("1-100 arası bir sayı tuttum. Tahmin edin:\n");
    do {
        printf("Tahmininiz: ");
        scanf("%d", &tahmin);
        deneme++;
        if (tahmin > hedef) printf("Daha küçük!\n");
        else if (tahmin < hedef) printf("Daha büyük!\n");
    } while (tahmin != hedef);
    printf("Tebrikler! %d denemede buldunuz!\n", deneme);
    return 0;
}
```

4.5. do-while Döngüsü

`do-while`, diğer döngülerden farklı olarak **çıkış kontrollü (post-test)** bir döngüdür. Gövde **en az bir kez** mutlaka çalıştırılır, koşul **sonra** kontrol edilir.

Sözdizimi:

```
do {
    // Döngü gövdesi - EN AZ 1 KEZ çalışır
} while (koşul);  // ← Sonunda noktalı virgül (;) zorunludur!
```

4.6. do-while vs while Farkı

```c
int x = 100;

// while: Koşul başta kontrol edilir → Gövde HİÇ ÇALIŞMAZ
while (x < 10) {
    printf("while: %d\n", x); // Bu satır ASLA çalışmaz
}

// do-while: Gövde önce çalışır → EN AZ 1 KEZ çalışır
do {
    printf("do-while: %d\n", x); // Bu satır 1 kez çalışır
} while (x < 10);
```

Çıktı: Sadece `do-while: 100` yazdırılır.

4.7. do-while: Girdi Doğrulama (Validation)

`do-while` en çok **kullanıcı girdisi doğrulama** senaryolarında tercih edilir. En az bir kez sormak gerektiği için idealdir:

```
1   int secim;
2   do {
3       printf("\n===== MENÜ =====\n");
4       printf("1. Ekle\n");
5       printf("2. Sil\n");
6       printf("3. Listele\n");
7       printf("4. Çıkış\n");
8       printf("Seçiminiz (1-4): ");
9       scanf("%d", &secim);
10
11      if (secim < 1 || secim > 4)
12          printf("Geçersiz seçim! Lütfen 1-4 arası girin.\n");
13  } while (secim < 1 || secim > 4);
```

BÖLÜM 5

Döngü Kontrol Kavramları ve Algoritma Desenleri

5.1. Sayaç Kontrollü Döngüler (Counter-Controlled)

Döngünün kaç kez çalışacağı önceden bellidir. Genellikle `for` ile tercih edilir.

```
// 1'den 100'e kadar sayıları yazdır
for (int i = 1; i <= 100; i++) {
    printf("%d ", i);
}

// 1'den n'e kadar sayıların toplamını hesapla
int toplam = 0;
for (int i = 1; i <= n; i++) {
    toplam += i;
}
printf("Toplam: %d\n", toplam);
```

5.2. Koşul Kontrollü Döngüler (Condition-Controlled)

Döngünün kaç kez çalışacağı önceden bilinmez; bir koşula bağlıdır. Genellikle `while` tercih edilir.

```
// Kullanıcı geçerli bir değer girene kadar tekrarla
int girdi;
do {
    printf("1-10 arası bir sayı girin: ");
    scanf("%d", &girdi);
} while (girdi < 1 || girdi > 10);

// Hata payı yeterince küçülene kadar hesapla
while (fabs(hata) > 0.0001) {
    // Newton-Raphson iterasyonu
    tahmin = tahmin - f(tahmin) / f_turev(tahmin);
    hata = f(tahmin);
}
```

5.3. Sentinel (Nöbetçi) Kontrollü Döngüler

Özel bir **kaçış değeri (sentinel)** girilene kadar döngü çalışmaya devam eder. Dosya sonu (EOF), −1, 0 gibi değerler sentinel olarak kullanılır.

```c
int sayi, toplam = 0, adet = 0;
printf("Sayı girin (-1 ile çıkış): ");
scanf("%d", &sayi);

while (sayi != -1) {        // -1 = sentinel değer
    toplam += sayi;
    adet++;
    printf("Sayı girin (-1 ile çıkış): ");
    scanf("%d", &sayi);
}
printf("Ortalama: %.2f\n", (double)toplam / adet);
```

5.4. Algoritma Deseni 1: Akümülatör (Toplama / Çarpma)

Döngü boyunca bir değişkende değer biriktirme işlemidir. Matematikteki **Σ** (sigma) ve **Π** (pi) notasyonlarına karşılık gelir.

```
// Σ Toplama: 1+2+3+...+n
int toplam = 0;  // Toplam için etkisiz eleman: 0
for (int i = 1; i <= n; i++) {
    toplam += i;
}

// Π Çarpma (Faktöriyel): n! = 1×2×3×...×n
long long fakt = 1;  // Çarpım için etkisiz eleman: 1
for (int i = 1; i <= n; i++) {
    fakt *= i;
}
```

⚠️ **Dikkat:** Akümülatörün başlangıç değeri çok önemlidir. Toplam için `0`, çarpım için `1` olmalıdır.

5.5. Algoritma Deseni 2: Min / Max Bulma

Bir veri kümesindeki en büyük veya en küçük değeri bulmak için kullanılır. İlk değer referans (geçici şampiyon) olarak atanır, sonraki değerler ile karşılaştırılır.

```c
int n, sayi;
printf("Kaç sayı gireceksiniz? ");
scanf("%d", &n);

printf("1. sayı: ");
scanf("%d", &sayi);
int max = sayi;  // İlk sayıyı şampiyon ilan et
int min = sayi;

for (int i = 2; i <= n; i++) {
    printf("%d. sayı: ", i);
    scanf("%d", &sayi);
    if (sayi > max) max = sayi;  // Yeni şampiyon
    if (sayi < min) min = sayi;  // Yeni minimum
}
printf("Max: %d, Min: %d\n", max, min);
```

5.6. Algoritma Deseni 3: Arama (Search) ve Bayrak (Flag)

Kullanıcıdan alınan sayılar arasında belirli bir değerin varlığını kontrol etme. Bulunduğunda `flag` değişkeni `1` yapılır:

```c
int aranan = 67;
int bulundu = 0;  // Flag: 0 = bulunamadı, 1 = bulundu
int n, sayi;

printf("Kaç sayı gireceksiniz? ");
scanf("%d", &n);

for (int i = 1; i <= n; i++) {
    printf("%d. sayı: ", i);
    scanf("%d", &sayi);
    if (sayi == aranan) {
        bulundu = 1;
        printf("%d bulundu!\n", aranan);
        break;  // Bulunca döngüden çık → performans kazancı
    }
}
if (!bulundu) {
    printf("%d bulunamadı.\n", aranan);
}
```

5.7. Algoritma Deseni 4: İterasyon (Sayısal Yakınlaştırma)

Matematiksel formülleri bilgisayarda adım adım çözmek için kullanılır. Her adımda sonuç gerçek değere daha çok yakınsar.

```
// Newton-Raphson ile karekök hesaplama: √25
double x = 25.0;
double tahmin = x / 2.0;  // İlk tahmin

for (int i = 1; i <= 10; i++) {
    tahmin = (tahmin + x / tahmin) / 2.0;
    printf("%2d. iterasyon: %.10f\n", i, tahmin);
}
// 10 adımda tahmin ≈ 5.0000000000'a yakınsar
```

Bu teknik sensör kalibrasyonu, kontrol sistemleri ve optimizasyon problemlerinde sıkça kullanılır.

layout: intro

BÖLÜM 6

break ve continue Deyimleri

6.1. break Deyimi

`break`, içinde bulunduğu en yakın döngüyü veya switch yapısını anında sonlandırır. Program akışı döngüden sonraki ilk satıra atlar.

```
1   // İlk negatif sayıyı bul ve dur
2   int sayi;
3   for (int i = 1; i <= 10; i++) {
4       printf("%d. sayı: ", i);
5       scanf("%d", &sayi);
6       if (sayi < 0) {
7           printf("İlk negatif sayı: %d (sıra: %d)\n", sayi, i);
8           break;  // Döngüden hemen çık
9       }
10  }
11  // Program buradan devam eder
```

⚠️ İç içe döngülerde: `break` yalnızca en içteki döngüyü kırar. Dış döngü etkilenmez.

6.2. continue Deyimi

`continue`, döngünün **o anki iterasyonunun kalan kısmını atlayıp** bir sonraki iterasyona geçirir.

```c
// 1'den 20'ye kadar olan sayılardan sadece tek sayıları topla
int toplam = 0;
for (int i = 1; i <= 20; i++) {
    if (i % 2 == 0) {
        continue;  // Çift sayıysa bu turu atla, sonraki i'ye git
    }
    toplam += i;  // Sadece tek sayılar buraya ulaşır
}
printf("Tek sayıların toplamı: %d\n", toplam);
// Çıktı: Tek sayıların toplamı: 100
```

- `for` döngüsünde: **güncelleme** adımına atlar
- `while` / `do-while` döngüsünde: **koşul kontrolüne** atlar

6.3. break ve continue Karşılaştırma Tablosu

Özellik	`break`	`continue`
Etki	Döngüyü **tamamen sonlandırır**	Sadece **mevcut turu atlar**
Sonrası	Döngüden **sonraki** satıra gider	Döngünün **güncelleme/koşul** adımına gider
Kullanım	Arama bulunduğunda, hata durumunda	Filtreleme, belirli değerleri atlama
iç içe döngü	Sadece **en yakın** döngüyü etkiler	Sadece **en yakın** döngüyü etkiler

6.4. Sonsuz Döngü + break Deseni

Gömülü sistemlerde ve sürekli çalışan uygulamalarda sıkça kullanılır:

```c
while (1) {  // veya for (;;)
    int sicaklik = sicaklik_oku();

    if (sicaklik > 100) {
        printf("ALARM: Aşırı sıcaklık! Sistem kapatılıyor.\n");
        break;  // Acil çıkış
    }

    printf("Sıcaklık: %d°C - Normal\n", sicaklik);
    // ... kontrol döngüsü devam eder
}
```

BÖLÜM 7

Uygulamalar

7.1. Uygulama 1: Sayısal Analiz - Taylor Serisi

e^x değerinin Taylor serisi açılımı ile hesaplanması:

```c
#include <stdio.h>
#include <math.h>

int main() {
    double x;
    int n;
    printf("x değeri: "); scanf("%lf", &x);
    printf("Terim sayısı: "); scanf("%d", &n);

    double sonuc = 1.0, terim = 1.0;
    for (int i = 1; i <= n; i++) {
        terim *= x / i;    // x^i / i! (önceki terimden türet)
        sonuc += terim;
        printf("%2d. terim sonrası: %.10f\n", i, sonuc);
    }
    printf("Hesaplanan: %.10f\n", sonuc);
    printf("Gerçek exp(%g): %.10f\n", x, exp(x));
    return 0;
}
```

7.2. Uygulama 2: Leibniz ile π Hesaplama

```c
// Leibniz formülü: π/4 = 1 - 1/3 + 1/5 - 1/7 + ...
double pi_4 = 0.0;
int isaret = 1;

for (int i = 0; i < n; i++) {
    pi_4 += isaret * (1.0 / (2 * i + 1));
    isaret = -isaret;  // İşareti her adımda değiştir

    if ((i + 1) % 10000 == 0) {
        printf("%6d terim → π ≈ %.10f\n", i + 1, 4 * pi_4);
    }
}
printf("Hesaplanan π = %.10f\n", 4 * pi_4);
printf("math.h π     = %.10f\n", M_PI);
```

Seri yavaş yakınsar: 100.000 terimde bile sadece ~5 basamak doğruluk sağlanır.

7.3. Uygulama 3: Dizi İşlemleri - Ters Çevirme ve Sıralama

```
// Diziyi ters çevirme (Swap yöntemi)
for (int i = 0; i < n / 2; i++) {
    int gecici = dizi[i];
    dizi[i] = dizi[n - 1 - i];
    dizi[n - 1 - i] = gecici;
}
```

```
// Bubble Sort - O(n²) sıralama algoritması
for (int i = 0; i < n - 1; i++) {
    for (int j = 0; j < n - 1 - i; j++) {
        if (dizi[j] > dizi[j + 1]) {
            int gecici = dizi[j];
            dizi[j] = dizi[j + 1];
            dizi[j + 1] = gecici;
        }
    }
}
```

7.4. Uygulama 4: Fibonacci Serisi

Her sayı kendinden önceki iki sayının toplamıdır: 0, 1, 1, 2, 3, 5, 8, 13, 21...

```c
int n;
printf("Kaç terim? "); scanf("%d", &n);

long long onceki = 0, simdiki = 1, sonraki;
printf("Fibonacci Serisi:\n");
printf("%lld %lld ", onceki, simdiki);

for (int i = 3; i <= n; i++) {
    sonraki = onceki + simdiki;
    printf("%lld ", sonraki);
    onceki = simdiki;    // Kaydırma (shift)
    simdiki = sonraki;
}
printf("\n");
```

Dikkat: `long long` kullanılmazsa büyük terimlerde taşma (overflow) olur.

7.5. Uygulama 5: Sinyal İşleme - Sinyal Üretimi ve Filtreleme

```c
#define N 100
#define FS 1000.0  // Örnekleme frekansı (Hz)

double sinyal[N], filtreli[N];
double frekans = 50.0, genlik = 220.0;
double dt = 1.0 / FS;

// 1. Sinüs sinyali üret
for (int i = 0; i < N; i++) {
    sinyal[i] = genlik * sin(2 * M_PI * frekans * i * dt);
}

// 2. Hareketli ortalama filtresi (pencere = 5)
int w = 2; // yarı pencere
for (int i = 0; i < N; i++) {
    double top = 0; int cnt = 0;
    for (int j = -w; j <= w; j++) {
        if (i+j >= 0 && i+j < N) { top += sinyal[i+j]; cnt++; }
    }
    filtreli[i] = top / cnt;
```

7.6. Uygulama 5 (devam): Sinyal Analizi

```c
// 3. Ortalama, RMS ve Tepe değer hesaplama
double ort = 0.0, rms = 0.0, tepe = 0.0;
for (int i = 0; i < N; i++) {
    ort += sinyal[i];
    rms += sinyal[i] * sinyal[i];
    if (fabs(sinyal[i]) > tepe) tepe = fabs(sinyal[i]);
}
ort /= N;
rms = sqrt(rms / N);  // Root Mean Square

printf("Ortalama : %.4f\n", ort);
printf("RMS      : %.4f\n", rms);
printf("Tepe     : %.4f\n", tepe);
printf("Crest F. : %.4f\n", tepe / rms);  // Tepe faktörü
```

7.7. Uygulama 5 (devam): Frekans Tespiti

Sıfır geçişlerini (zero-crossing) sayarak sinyalin frekansını yazılımsal olarak tahmin etme:

```c
int gecis = 0, ilk = -1, son = -1;
for (int i = 1; i < N; i++) {
    // Negatiften pozitife geçiş anını yakala
    if (sinyal[i-1] <= 0 && sinyal[i] > 0) {
        gecis++;
        if (ilk == -1) ilk = i;
        son = i;
    }
}
if (gecis >= 2) {
    double periyot = (son - ilk) * dt / (gecis - 1);
    printf("Tahmin edilen frekans: %.2f Hz\n", 1.0 / periyot);
    printf("Gerçek frekans: %.2f Hz\n", frekans);
}
```

Döngü Seçim Rehberi - Özet Tablosu

Soru	Cevap	Döngü Türü
Kaç kez tekrarlanacak belli mi?	Evet	`for`
Koşul sağlandığı sürece devam mı?	Evet	`while`
En az 1 kez çalışmalı mı?	Evet	`do-while`
Özel bir çıkış değeri var mı?	Evet	`while` + sentinel
Sürekli çalışacak bir sistem mi?	Evet	`while(1)` + `break`

Sık Yapılan Döngü Hataları

1. Sonsuz döngü: Güncelleme unutulması → `while(i < 10)` ama `i++` yok
2. Off-by-one: `<` yerine `<=` veya tersi kullanımı
3. Noktalı virgül hatası: `for(i=0; i<10; i++);` → Gövde boş döner!
4. Yanlış koşul: `while(x = 5)` atama yapar, `while(x == 5)` olmalı
5. do-while sonunda `;` unutma: `} while(koşul)` → derleme hatası

```c
// HATA: for'dan sonra noktalı virgül — Gövde boş çalışır!
for (int i = 0; i < 10; i++);  // ← Bu ; gövdeyi boşaltır
{
    printf("%d ", i);  // Sadece 1 kez çalışır (i=10 ile)
}
```

UYGULAMALAR

Aşağıdaki 10 uygulama, döngü konularını pekiştirmek için tasarlanmıştır.

Uygulama 1: Asal Sayı Kontrolü

Kullanıcıdan bir sayı alıp asal olup olmadığını kontrol eden program:

```c
#include <stdio.h>

int main() {
    int sayi, asal = 1;
    printf("Bir sayi giriniz: ");
    scanf("%d", &sayi);

    if (sayi < 2) asal = 0;

    for (int i = 2; i * i <= sayi; i++) {
        if (sayi % i == 0) {
            asal = 0;
            break; // Bölen bulundu, devamına bakmaya gerek yok
        }
    }

    if (asal)
        printf("%d bir asal sayidir.\n", sayi);
    else
        printf("%d asal degildir.\n", sayi);
```

Uygulama 2: 1'den N'e Kadar Asal Sayıları Listeleme

```c
#include <stdio.h>

int main() {
    int n;
    printf("Ust sinir (N): ");
    scanf("%d", &n);

    printf("1-%d arasi asal sayilar:\n", n);
    for (int sayi = 2; sayi <= n; sayi++) {
        int asal = 1;
        for (int i = 2; i * i <= sayi; i++) {
            if (sayi % i == 0) {
                asal = 0;
                break;
            }
        }
        if (asal) printf("%d ", sayi);
    }
    printf("\n");
    return 0;
```

Kullanılan Kavramlar: İç içe `for`, `break`, flag

Uygulama 3: Mükemmel Sayı Kontrolü

Bir sayının kendisi hariç pozitif bölenlerinin toplamı kendisine eşitse mükemmeldir (6, 28, 496...):

```c
#include <stdio.h>

int main() {
    int sayi;
    printf("Bir sayi giriniz: ");
    scanf("%d", &sayi);

    int bolen_toplami = 0;
    for (int i = 1; i < sayi; i++) {
        if (sayi % i == 0) {
            bolen_toplami += i;
            printf("%d ", i);  // Bölenleri yazdır
        }
    }

    printf("\nBolenlerin toplami: %d\n", bolen_toplami);
    if (bolen_toplami == sayi)
        printf("%d bir MUKEMMEL sayidir!\n", sayi);
    else
        printf("%d mukemmel sayi degildir.\n", sayi);
```

Uygulama 4: Üs Alma (Kuvvet Hesaplama)

`pow()` fonksiyonu kullanmadan, döngü ile taban^üs hesaplama:

```c
#include <stdio.h>

int main() {
    int taban, us;
    printf("Tabani giriniz: ");
    scanf("%d", &taban);
    printf("Usse giriniz: ");
    scanf("%d", &us);

    long long sonuc = 1; // Çarpım akümülatörü (etkisiz eleman = 1)
    for (int i = 0; i < us; i++) {
        sonuc *= taban;
    }
    printf("%d ^ %d = %lld\n", taban, us, sonuc);
    return 0;
}
```

Kullanılan Kavramlar: `for`, çarpma akümülatörü, `long long` taşma önlemi

Uygulama 5: Girdi Doğrulama ile Not Ortalaması Hesaplama

`do-while` ile geçersiz girdiyi reddetme ve `while` ile sentinel kontrollü döngü:

```c
#include <stdio.h>

int main() {
    int not_degeri, toplam = 0, adet = 0;

    printf("Notlari giriniz (cikmak icin -1):\n");
    while (1) {
        // Girdi doğrulama: 0-100 arası veya -1 olmalı
        do {
            printf("Not: ");
            scanf("%d", &not_degeri);
            if (not_degeri != -1 && (not_degeri < 0 || not_degeri > 100))
                printf("Gecersiz! 0-100 arasi veya -1 girin.\n");
        } while (not_degeri != -1 && (not_degeri < 0 || not_degeri > 100));

        if (not_degeri == -1) break;  // Sentinel değer → çık

        toplam += not_degeri;
        adet++;
    }
```

Uygulama 6: Sayının Tersini Alma (Basamak Ters Çevirme)

```c
#include <stdio.h>

int main() {
    int sayi, ters = 0;
    printf("Bir tam sayi giriniz: ");
    scanf("%d", &sayi);

    int orijinal = sayi;
    while (sayi != 0) {
        int basamak = sayi % 10;   // Son basamağı al
        ters = ters * 10 + basamak; // Ters sayıya ekle
        sayi /= 10;                 // Son basamağı at
    }
    printf("%d sayisinin tersi: %d\n", orijinal, ters);

    // Palindrom kontrolü (bonus)
    if (orijinal == ters)
        printf("%d bir palindrom sayidir!\n", orijinal);
    return 0;
}
```

Kullanılan Kavramlar: `while`, mod ve tamsayı bölme, değişken takası mantığı

Uygulama 7: Elmas (Baklava) Deseni Çizimi

İç içe for döngüleri ile simetrik şekil çizimi:

```c
#include <stdio.h>

int main() {
    int n;
    printf("Boyut (tek sayi giriniz): ");
    scanf("%d", &n);

    int yari = n / 2;

    // Üst yarı (piramit)
    for (int i = 0; i <= yari; i++) {
        for (int j = 0; j < yari - i; j++) printf(" ");
        for (int j = 0; j < 2 * i + 1; j++) printf("*");
        printf("\n");
    }

    // Alt yarı (ters piramit)
    for (int i = yari - 1; i >= 0; i--) {
        for (int j = 0; j < yari - i; j++) printf(" ");
        for (int j = 0; j < 2 * i + 1; j++) printf("*");
```

Uygulama 8: EBOB ve EKOK Hesaplama (Öklid Algoritması)

```c
#include <stdio.h>

int main() {
    int a, b;
    printf("Iki sayi giriniz: ");
    scanf("%d %d", &a, &b);

    int orijinal_a = a, orijinal_b = b;

    // Öklid Algoritması — while döngüsü ile EBOB
    while (b != 0) {
        int kalan = a % b;
        a = b;
        b = kalan;
    }
    int ebob = a;
    int ekok = (orijinal_a * orijinal_b) / ebob;

    printf("EBOB(%d, %d) = %d\n", orijinal_a, orijinal_b, ebob);
    printf("EKOK(%d, %d) = %d\n", orijinal_a, orijinal_b, ekok);
```

Kullanılan Kavramlar: `while`, Öklid algoritması, iteratif güncelleme

Uygulama 9: Çift Sayıları Atla, Tek Sayıları Topla (continue)

`continue` deyiminin pratik kullanımını gösteren örnek:

```c
#include <stdio.h>

int main() {
    int n;
    printf("Ust sinir (N): ");
    scanf("%d", &n);

    int tek_toplam = 0;
    printf("Islenen tek sayilar: ");
    for (int i = 1; i <= n; i++) {
        if (i % 2 == 0) {
            continue; // Çift sayıları atla, döngünün sonraki turuna geç
        }
        // Buraya sadece tek sayılar ulaşır
        printf("%d ", i);
        tek_toplam += i;
    }
    printf("\n1-%d arasi tek sayilarin toplami: %d\n", n, tek_toplam);
    return 0;
}
```

Uygulama 10: Basit Hesap Makinesi (Menü Döngüsü)

Tüm döngü ve kontrol yapılarını bir arada kullanan bütünleşik uygulama:

```c
#include <stdio.h>

int main() {
    int secim;
    double a, b, sonuc;

    do {
        printf("\n====== HESAP MAKINESI ======\n");
        printf("1. Toplama\n");
        printf("2. Cikarma\n");
        printf("3. Carpma\n");
        printf("4. Bolme\n");
        printf("5. Faktoriyel\n");
        printf("0. Cikis\n");
        printf("Seciminiz: ");
        scanf("%d", &secim);

        if (secim == 0) break;

        if (secim >= 1 && secim <= 4) {
```

Uygulama 10 (devam)

```
        switch (secim) {
            case 1: printf("Sonuc: %.2f\n", a + b); break;
            case 2: printf("Sonuc: %.2f\n", a - b); break;
            case 3: printf("Sonuc: %.2f\n", a * b); break;
            case 4:
                if (b != 0)
                    printf("Sonuc: %.2f\n", a / b);
                else
                    printf("HATA: Sifira bolme!\n");
                break;
            case 5: {
                int n;
                printf("Sayi girin: ");
                scanf("%d", &n);
                long long fakt = 1;
                for (int i = 1; i <= n; i++) fakt *= i;
                printf("%d! = %lld\n", n, fakt);
                break;
            }
            default:
```

Kullanılan Kavramlar: `do-while`, `switch-case`, `for`, `break`, `if-else`, menü sistemi

Teşekkürler

Soru ve Cevaplar

Marmara Üniversitesi – Bilgisayar Programlama Dersi Hafta 5: Döngüler (for, while, do-while, break, continue)